

PoC ACL — conector CRS → Opera Cloud (bajada de reservas)

Objetivo

Construir el camino de **bajada** de una reserva —creada, modificada o cancelada— desde un CRS simulado hasta el tenant de pruebas de Opera Cloud, siguiendo el HLA *CRS-PMS Integration - Solution* (wiki, Support Domain / Integration Subdomain).

AC: tener el **coste real** de desarrollar un conector contra la API de un tercero. Por eso el esfuerzo se registra desde el primer día en [cost-log.md](#), y el del conector OHIP (`pms-integration-service`) se mide por separado del resto.

Alcance

Dentro

HLA	Proceso / capacidad
#1 (F001, F002, F014)	Proyectar Reserva: alta y modificación como el mismo <i>upsert</i> de estado, con perfil de huésped, desglose diario, referencias externas y cobros
#2 (F003)	Proyectar Cancelación, sin apunte de penalización en el folio
#3 (F004)	Proyectar Interlocutor: perfiles Travel Agent / Company en Opera desde el maestro de interlocutores
#9 (F009)	Mapeado de códigos: diccionario cadena + propiedad, versionado, con aprobación humana y propuesta por un agente
F012 (parcial)	Suspensión por causa y reanudación en bloque al resolverla
Transversal	MCP en cada servicio con operativa, notificaciones a las personas
F016	Auditoría (<code>audit-service</code>): las acciones declaradas <code>@Audited</code> en integraciones y mapeado — hechas o rechazadas, por consola, REST o agente— emiten <code>AuditedAction</code> por outbox; el servicio las materializa, inmutables, y la consola las lista con búsqueda libre y filtros por fecha, hotel, usuario, acción, servicio y resultado. Desde el PR #147 también las operaciones de recepción sobre una estancia (check-in, check-out, cambio de habitación, cargos, cobros, llave, wifi, firma, kárdex, walk-in, no-show...) y las del CRS sobre una reserva, con quién las hizo —la persona del token o el agente por ella—; la ficha de la reserva enseña su «Historial»
Recepción	El front office del hotel (H13, H14 y siguientes): estancias alimentadas por lo que Opera tiene; check-in, check-out y no-show que registra el PMS; cargos al folio de Opera; walk-in que reserva el CRS; kárdex con el MDM; avisos de recepción de un cliente (maestro Salesforce), de una reserva o de una agencia (servicio <code>notices</code> , PR #145)
HLA CRM-MDM F001–F005 (H11)	Maestro de clientes: resolver la identidad de cada pasajero al proyectar, provisional si no hay certeza, limpieza y fusión en Salesforce , supervivencia en el MDM y propagación del código al perfil de Opera

Fuera: todo lo que sube del PMS al CRS (OOO, no-show, salida anticipada, cupo, streaming), conciliación, backfill (#10), recap, penalización en el folio, ciclo de vida completo de la integración (#8, #13), read model de causas (`integration-query-service`; se usa la vista del motor). Del HLA de CRM-MDM (H11), fuera: consentimiento y derecho al olvido (F006, F007), carga inicial del histórico (F008), gobierno de reglas desde

una UI (F009), deshacer una fusión (CM-R7), fidelización y el flujo PMS → MDM del AF (F013, F018, F019), y que el perfil de Opera lleve los datos del golden record y no solo su código.

Decisiones tomadas

- **Sin CDC.** Controlamos `booking`, así que el aviso de cambio es un **evento de dominio** publicado por **outbox** en la misma transacción. Como en el HLA, el evento es ligero —`bookingId`, `hotel`, `version`— y el dato se relee.
- **Property APIs de OHIP** (`rsv`, `crm`, catálogo de configuración), no `resnotif`. Desvía del HLA: `resnotif` no revalida precio ni disponibilidad y `rsv` puede que sí. Es una incógnita a cerrar en la PoC (junto a **R19**) y se anota como tal en las conclusiones.
- **Huéspedes dentro de la reserva.** El perfil de huésped en Opera se crea con los datos que trae la reserva (F001, R11); no hay CRM de clientes en esta PoC.
- **Interlocutores en un maestro aparte (partners)**, que simula el maestro de interlocutores del lado ERP. La reserva de `booking` solo lleva el **código** del interlocutor. El proceso #3 lee **directamente del maestro**, que es la alternativa de **R38**.
- **Motor:** `EventConductor`, el que ya corre en el clúster. La suspensión es un paso `WAIT_FOR_MESSAGE` y la reanuda un `MessageReceived` (detalle más abajo).
- **Orden de aplicación:** guarda de secuencia *compare-and-set* contra un **UDF** de la reserva en Opera (R18, R28). La versión la da `booking`.
- **El LLM nunca está en el camino del dato.** Solo propone mapeados; nada entra en vigor sin aprobación humana.
- **No se escribe nada en el tenant de Opera** (decisión del 2026-09-22). El conector se construye contra `opera-mock`, un doble de OHIP que implementa las mismas rutas y formatos de las Property APIs que usamos: token OAuth, catálogo, perfiles, reservas con UDF y cancelación. Cambiar al tenant real es cambiar la URL y las credenciales. Validar contra el tenant (R18, R19, revalidación de `rsv`) queda como paso posterior a la PoC. *Revisada:* desde H12 el conector escribe en el tenant real (XMAR), y el 2026-09-24 `opera-mock` sale del despliegue de `ec1` —menús, gateway y manifiesto—. Queda solo como doble de la batería punta a punta local (`e2e/poc-acl-local`), que inyecta fallos y agota cupo, cosas que no se hacen contra el tenant.
- **Entre servicios, órdenes por Kafka; HTTP solo para consultar.** Una orden a otro servicio sale por el outbox del emisor y la consume un consumidor idempotente con inbox. HTTP síncrono queda para consultas y pantallas que esperan la respuesta. Qué va por dónde y lo que aún falta mover: [llamadas-sincronas.md](#).
- **La suspensión por causa, sin ciclos en el grafo.** El motor no admite ciclos, así que «volver a Preparar» se hace así: 1. `Preparar` devuelve las carencias y `mapping-service` registra cada una como causa, con los procesos que esperan detrás. 2. El proceso espera en un único `WAIT_FOR_MESSAGE` correlacionado por su propia clave. 3. Cuando un proceso se queda sin causas abiertas, `mapping-service` le envía el mensaje. 4. El proceso reanudado **relanza una instancia nueva**, que vuelve a leer la reserva.

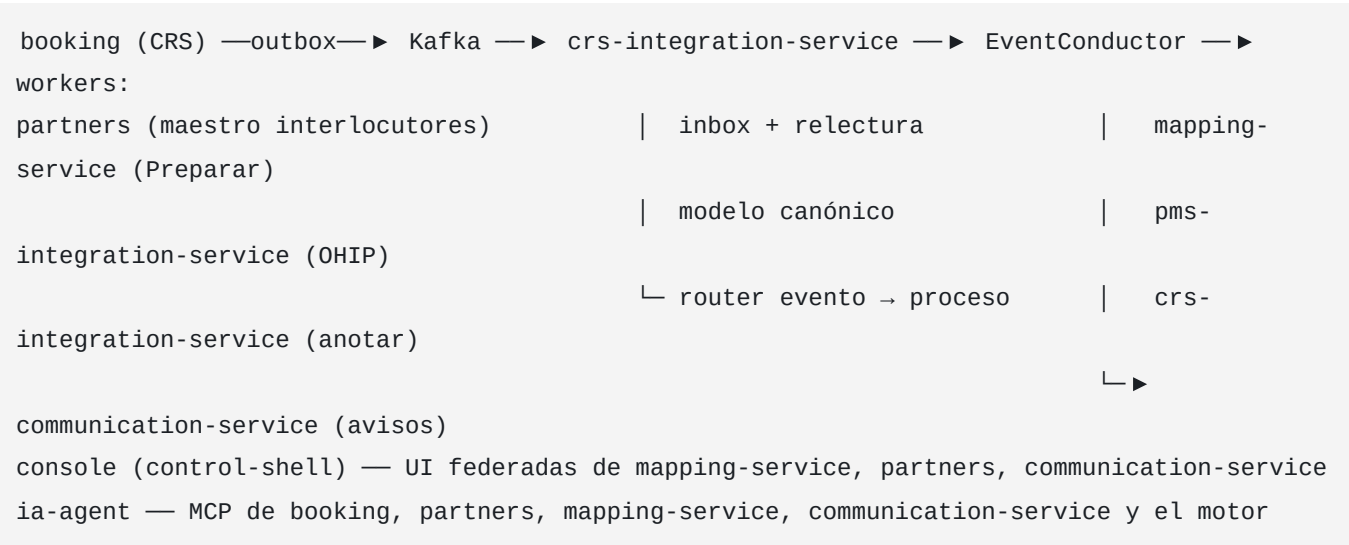
Encaja con «una instancia por evento» del HLA: una instancia de más es inofensiva. Un rechazo determinista de Opera usa el mismo camino, como una causa más.

- **Tenant real de Opera (H12, 23-09-2026).** Se levanta la decisión anterior: en OHIP UAT se escriben **solo reservas y perfiles de huésped** (con sus cancelaciones), primero en **XMAR** (XMU si XMAR no sirviera). Los **interlocutores son de Opera**: se leen y se importan al ERP por su `CorporateId`, nunca se crean en Opera (`PARTNERS_OWNED_BY_PMS`). Lo que el tenant aún no tiene configurado —interfaces

para referencias en perfiles, cajero del usuario de integración para depósitos— queda tras interruptores (OPERA_PROFILE_REFERENCES, OPERA_POST_DEPOSITS).

- **El maestro de clientes es nuestro; Salesforce limpia (H11, HLA CRM-MDM).** `customer-mdm-service` guarda el golden record y resuelve la identidad; Salesforce recibe los clientes como `Contact` (upsert por el campo externo `MDM_Id__c`), sus reglas de duplicados proponen y un *steward* fusiona. La vuelta es un Platform Event propio, `ClienteConsolidado__e`, por la **Pub/Sub API**, con *polling* por `queryAll` como red de seguridad.
- **Sin Apex.** La org es *Base Edition*: no admite desplegar Apex. El evento lo publica un **Flow** *before delete*, y ahí `MasterRecordId` aún está vacío: el evento dice qué cliente se fue y el MDM lee el superviviente del contacto borrado (`queryAll`). Otro Flow *before save* impide que una fusión reasigne `MDM_Id__c`. Los Flows se despliegan como borrador en producción: `deploy.py` los activa.
- **Match solo con certeza.** Documento, o email con el mismo nombre; el titular que también es huésped de una habitación es el mismo cliente. Lo demás es un provisional nuevo: fundir a dos personas distintas es peor que un duplicado, que es para lo que está la limpieza.
- **El código viaja por los raíles existentes.** Una fusión no reescribe la reserva del CRS: el MDM guarda qué pasajero de qué reserva es qué cliente y pide a `crs-integration-service` que la **proyecte de nuevo**. `ensure-guest-profile` vuelve a resolver y escribe en el perfil de Opera la referencia externa `CRM` (el `CRM_GUID` del AF) del superviviente; la reserva, ya en esa versión, no se toca. Desvía del HLA, que re-estampa en el CRS: aquí no hace falta tocar `booking`.
- **El MDM no es una puerta.** Si no responde, el perfil se escribe sin código y la venta sigue.

Piezas



Servicio	Nuevo / cambia	Responsabilidad	UI	MCP
booking	Cambia	CRS simulado: modelo de reserva real, versión, eventos de dominio, outbox, lectura completa, anotar referencia PMS	Sí	Sí (ampliado)
partners	Nuevo	Maestro de interlocutores: agencias, turoperadores y companies con datos fiscales, facturación y modalidad Front / No Front	Sí	Sí
integration-model	Nuevo (librería)	Modelo de negocio canónico y contratos de eventos. Ningún servicio fuera de los extremos conoce booking ni Opera	—	—

Servicio	Nuevo / cambia	Responsabilidad	UI	MCP
crs-integration-service	Nuevo	ACL del lado CRS: inbox con deduplicación, relectura, traducción al modelo canónico, router evento → proceso, worker «anotar en el CRS»	No	No
mapping-service	Nuevo	Diccionario de equivalencias y de identificadores (perfiles de interlocutor), worker Preparar , causas, aprobación, señal de reanudación, petición de propuesta al agente	Sí	Sí
pms-integration-service	Nuevo	El conector. ACL contra OHIP Property APIs: auth, catálogo, perfiles, reserva, cobros, cancelación, clasificación de errores	No	No
opera-mock	Nuevo (doble)	Simula OHIP Property APIs con estado en memoria y una UI de solo lectura para ver qué ha «llegado a Opera»	Sí	No
communication-service	Nuevo	Envío de notificaciones: plantillas, destinatarios, canal email por el relay postfix , histórico	Sí	Sí
ec-definitions	Cambia	Definiciones proyectar-reserva, proyectar-cancelacion, proyectar-interlocutor	—	—
ia-control-plane	Configuración	Alta de los MCP nuevos y del agente de mapeado	—	—
front-office	Nuevo (H13); cambia (H14)	El front office del hotel (check-in, en casa, check-out, folios), traído de la demo de Mateu. Desde H14 consume el PMS : la integración pms-fo le manda el catálogo de Opera y cada reserva tal como Opera la tiene (front-office-commands), y toma el kárdex del MDM directamente (customers)	Sí	No
customer-mdm-service	Nuevo (H11)	Maestro de clientes: golden record, resolución de identidad, proyección a Salesforce, suscripción a ClienteConsolidado__e , supervivencia y propagación del código; metadatos de Salesforce en salesforce/	Sí	Sí

Los adaptadores (crs- y pms-integration-service) no tienen UI ni MCP, como en el HLA: traducen, y no tienen operativa propia que enseñar.

Detalle por pieza

booking como Rumbo

- **Modelo**
- Cabecera: hotel, canal, código de interlocutor, bono externo, fechas, moneda, estado y **versión** monótona.
- Habitaciones: tipo, tarifa y régimen propios del CRS, ocupación y precio por noche.
- Huéspedes por habitación, con el titular marcado.
- Cobros: tipo, forma de pago, importe, fecha e id estable.
- Los códigos son de estilo CRS, deliberadamente distintos de los de Opera, para que el ACL tenga trabajo real.
- **Eventos**: BookingCreated, BookingModified y BookingCancelled por medio de AggregateRoot.send() / popEvents(). Una tabla outbox se escribe en la misma transacción y un

relay la publica a Kafka.

- **API:** lectura completa de la reserva (hace de «relectura por JDBC») y comando para anotar el `reservationId` de Opera.
- Adaptar la UI de Mateu (listas anidadas), el MCP y `api-mcp/.../bookings-openapi.yaml`.
- Alinear `shared` con la versión del motor desplegado (hoy 2.14.1 frente a 2.16.5).

partners

- CRUD de interlocutores con su tipo (Travel Agent, Source, Company), datos fiscales, dirección de facturación, modalidad de cobro y **versión**.
- Evento `PartnerChanged` por outbox, que dispara el proceso #3.
- MCP: listar, consultar, crear y modificar interlocutores, y relanzar su sincronización.

crs-integration-service

- **Inbox:** deduplica por `eventId` y agrupa avisos seguidos de la misma reserva.
- **Relectura** de `booking` y `partners`, y **traducción al modelo canónico**.
- **Router** (tabla evento → definición de proceso, en configuración): arranca el proceso con `ProcessCreationRequested`, uno por evento, con clave de correlación `hotel + localizador`.
- **Worker** «anotar la referencia del PMS en el CRS».

mapping-service

- **Diccionario** de cadena con excepciones por propiedad. Cubre: hotel, tipo de habitación, tarifa, régimen, estado, motivo de cancelación, forma de pago, canal / origen y tipo de interlocutor. Admite correspondencias que no son 1:1.
- **Correspondencia de identificadores:** interlocutor del CRS ↔ perfil de Opera, con la versión proyectada.
- **Worker Preparar:** resuelve **todas** las traducciones de una vez y devuelve el payload listo o la **lista completa de carencias**.
- **Causas:** cada carencia es una causa con clave `hotel/tipo/código`, con cuántos procesos esperan detrás y desde cuándo. Al aprobar una equivalencia se emite `MessageReceived`, que reanuda todos los procesos que esperaban esa causa.
- **Catálogos:** los del CRS salen de `booking`; los de Opera, de `pms-integration-service`, y se cachean.
- **Propuesta por agente**
- Cuando hay mapeado pendiente, la UI ofrece «**Pedir propuesta al agente**». También se puede pedir desde el chat de la consola.
- El agente usa el MCP de `mapping-service`: lee las carencias y ambos catálogos y **registra una propuesta** con una confianza por línea.
- La propuesta queda en estado *propuesta* y el administrador la revisa, corrige, aprueba o rechaza en la UI.
- Aprobar por MCP solo con la identidad del usuario y confirmación explícita. Nunca aprueba el agente por su cuenta.
- **Versionado:** cada aprobación crea una versión nueva, con autor y fecha.
- **MCP:** carencias y causas abiertas, catálogos, diccionario vigente, registrar propuesta y aprobar o rechazar (con la identidad del usuario).

pms-integration-service (el conector)

- **Autenticación OHIP:** token OAuth con caché y renovación, `x-app-key`, enterprise y hotel. Las credenciales van en un Secret y no en el código.
- **Catálogo** de la propiedad para el mapeado: tipos de habitación, tarifas, market / source, formas de pago, motivos de cancelación.
- **Perfiles** (`crm`): buscar por referencia externa antes de crear.
- Huésped (provisional con los datos del titular).
- Travel Agent / Company para el proceso #3.
- **Reserva** (`rsv`):
 - Buscar por el localizador del CRS inyectado como referencia externa.
 - Leer el UDF de secuencia y comparar con la versión que llega.
 - Crear o actualizar con el desglose diario, tarifa fija, perfiles enlazados, enrutamiento Front / No Front y los cobros como depósito referenciado a su id de origen.
 - Una versión rezagada **no se graba**.
- **Cancelación:** con el motivo mapeado, de forma idempotente (si ya estaba cancelada, no es error).
- **Clasificación de errores:**

Respuesta de OHIP	Qué hace el proceso
Timeout, 5xx, límite de caudal	Transitorio: reintento
4xx determinista	Causa y suspensión
Conflicto (la reserva ya existe)	Se lee y se actualiza

- **Para el front office** (H14, integración pms-fo), solo lecturas: la reserva entera por su id de Opera (con sus paquetes), el catálogo de la propiedad tal como lo lee un front office (tipos de habitación, tarifas con nombre, todos los paquetes, las habitaciones) y las reservas de una ventana con su `lastModifyDateTime`. Tras grabar o cancelar publica `pms-reservations`; el paso `project-stay` de «Proyectar estancia» relee la reserva, la pasa al modelo del front office y la manda por `front-office-commands`.
- **Pruebas** contra `opera-mock`, que reproduce las rutas y formatos de las especificaciones públicas de OHIP (`oracle/hospitality-api-docs`). No hay respuestas reales del tenant: no se le escribe nada.

communication-service

- Consume `NotificationRequested` de Kafka. Resuelve plantilla y destinatarios por tipo de aviso y hotel, envía por email a través del relay `postfix` y guarda el histórico.
- **Avisos de la PoC:**
 - Causa nueva: falta un mapeado o un interlocutor, con enlace a la UI de mapeado.
 - Propuesta del agente lista para revisar.
 - Proceso que sigue reintentando pasado el umbral (R14).
 - Rechazo determinista de Opera.
 - UI de histórico y de destinatarios. MCP: consultar avisos y reenviar.
 - La integración decide **qué** se notifica y **a quién**; el canal es cosa de este servicio (HLA, «Notificación a las personas»).

customer-mdm-service (H11)

- **Resolver identidad** (`POST /identities/resolve`): el conector la llama en `ensure-guest-profile` con el titular y los huéspedes de cada habitación. Idempotente por reserva y pasajero.

- **Proyección a Salesforce:** los clientes pendientes van como `Contact`; lo que Salesforce rechaza queda marcado y no se reintenta hasta que el cliente cambia.
- **Vuelta:** `ClienteConsolidado__e` por Pub/Sub API (gRPC, reanuda por `replay id`) y, cada minuto, `queryAll` de los contactos borrados. Los dos acaban en la misma bandeja, deduplicada por cliente.
- **Supervivencia:** gana, campo a campo, lo que el *steward* dejó en el contacto superviviente; si no lo hay, lo que el MDM tenía; si tampoco, lo del absorbido. El absorbido queda como alias.
- **Proxy de las modificaciones:** un hotel propone un cambio (`POST /customers/{id}/change-requests`) y el MDM lo abre en Salesforce como Case; nadie más escribe en Salesforce.
- **Proyección de Salesforce:** el MDM guarda los datos del contacto como golden record cuando Salesforce los cambia (a mano o al aprobar un Case) — `ClienteActualizado__e`, `CambioClienteResuelto__e`.
- **Eventos, no llamadas:** lo que el MDM decide de un cliente lo publica en el topic `customers` (por su outbox, con clave el cliente): `CustomerChanged` (golden record nuevo, o la decisión de un cambio propuesto) y `CustomersMerged` (fusión, con el superviviente). Cada evento lleva el golden record y las reservas del cliente. El MDM no llama a nadie: el **front office** toma el kárdex del topic (desde H14; antes lo relevaba pms-integration), y **crs-integration** proyecta de nuevo las reservas del cliente (origen `mdm-update-<cliente>-v<versión>` o `mdm-merge-<absorbido>`) en los hoteles con integración, para que Opera reescriba el perfil. Un sistema nuevo que necesite el cliente es un suscriptor más.
- **Xref:** dónde se conoce al cliente fuera del MDM (contacto de Salesforce, huésped del front office, perfiles de Opera). En los dos sentidos: `GET /customers/{id}` los lista, y `GET /customers?xref=OPERA:20538296` dice qué cliente es (el superviviente, si se fusionó); también como herramienta MCP (`findCustomerByXref`).
- **Salesforce** (`salesforce/`): campos en `Contact`, el Platform Event, dos Flows, un Permission Set para el usuario de integración, una regla de coincidencia amplia y una regla de duplicados que **registra** los posibles duplicados sin bloquear. `deploy.py` lo despliega con las credenciales del propio MDM (client credentials), activa los Flows y asigna el Permission Set.
- UI (golden records, consolidaciones) y MCP de solo consulta.

La integración pms-fo (H14)

La decisión (2026-09-27): cada dato viaja solo desde su maestro a sus consumidores, en cadena o por un hub. Las reservas van **CRS** → **PMS** → **front office**: no hay integración crs-fo. El front office consume el PMS.

- **Qué es.** Una integración por propiedad de Opera y su front office (`FrontOfficeIntegration` en `integrations-service`, con su máquina de estados y su proceso de alta `alta-integracion-fo`): 1. **Conexión:** Opera legible (token y la propiedad) y el front office responde. 2. **Catálogo:** el catálogo de la propiedad tal como lo lee un front office — tipos de habitación, tarifas con su nombre, todos los paquetes (también los que no se venden sueltos, como el BKF de XMAR, que llega dentro de una tarifa) y las habitaciones con su tipo — va al front office (`replace-catalogue`). La puerta se abre cuando el front office dice que tiene ese mismo. 3. **Backfill:** las reservas de la ventana — en casa o con llegada dentro del horizonte (`horizonDays`, 60 por defecto) —, una `proyectar-estancia` por reserva, a ritmo del backfill. 4. **Activación** por una persona: desde ahí fluyen los cambios. Cada puerta que necesita a alguien deja su aviso en la bandeja, que se cierra al pasarla.
- **Ámbito** (`scope`): `ALL`, todas las reservas de la propiedad (también las nacidas en Opera), o `CHAIN`, solo las que escribió la integración de la cadena (Custom Reference `EC-DEM01`).

- **Cómo llega una reserva.** Lo que la integración crs-pms escribe en Opera lo avisa el conector (`pms-reservations`) y la integración pms-fo, si está activa, arranca `proyectar-estancia`. Su paso lee la reserva de **Opera** y la manda al front office con los códigos de Opera; el front office la lee con el catálogo que tiene (el tipo de habitación, el régimen: «Pensión Desayuno Adulto», no BKF). El cliente, del MDM: qué cliente es el perfil de Opera (su `xref`) y su `golden record`.
- **Cómo se detecta un cambio hecho en Opera.** OHIP no tiene en la búsqueda de reservas un filtro «modificadas desde» (probado en el tenant: los parámetros de ese tipo se ignoran), y los *business events* son una cola que exige suscribir un sistema externo en la configuración de Opera — algo que esta integración no toca. Lo que sí da la búsqueda es el `lastModifyDateTime` de cada reserva. Así que la integración **sondea** (`FO_POLL`, 60 s): recorre las reservas de la ventana, 200 por página (XMAR: unas mil en 90 días, cinco páginas), y proyecta las modificadas en o después de su **cursor** (la última modificación ya proyectada), que avanza en la misma transacción que arranca los procesos.
- **Idempotencia.** Cada proceso lleva como clave la reserva de Opera y su `lastModifyDateTime` (o el id del evento): el motor no arranca dos veces la misma. El front office deduplica por `commandId`, y ordena por la versión de Opera: una más antigua que la que tiene no se aplica; la misma se reaplica (así llega un cambio del cliente que no tocó la reserva, como una fusión en el MDM).
- **Qué casa con lo que ya había.** Una estancia se reconoce por la reserva de Opera; si no, por el localizador del CRS (su referencia externa en el contexto de la ejecución); si no, por el walk-in que abrió recepción. Una reserva nacida en Opera abre la estancia `OP-<confirmación>`.
- **Lo que no llega de Opera.** Los acompañantes: Opera solo tiene al titular (el conector no escribe acompañantes). Si Opera no manda ninguno, la estancia conserva los que tenía.

Definiciones de proceso (`ec-definitions`)

- **proyectar-reserva** 1. Preparar (`mapping-service`). 2. Si hay carencias: aviso y un `WAIT_FOR_MESSAGE` correlacionado por la clave del proceso. Al reanudarse, **relanza una instancia nueva** que vuelve a leer la reserva (el grafo no admite ciclos). 3. Si no las hay: asegurar el perfil del huésped → grabar la reserva → anotar en el CRS.
- **proyectar-cancelacion**: esperar a que la reserva esté proyectada → preparar el motivo → cancelar en Opera → anotar en el CRS.
- **proyectar-estancia** (H14): releer de Opera la reserva y grabarla en el front office como estancia. La arranca la integración pms-fo: por el evento `pms-reservations` (lo que la integración escribe en Opera), por su sondeo (lo que cambia en Opera por otras vías) y por su backfill.
- **alta-integracion-fo** (H14): el alta de la integración PMS → front office, por puertas: conexión (Opera y front office), catálogo del PMS en el front office, backfill, activación.
- **proyectar-interlocutor**: preparar → asegurar los perfiles → anotar la correspondencia → señal de reanudación para las reservas que esperaban ese interlocutor.
- **Política de reintento en los pasos externos**: backoff acotado sin límite de intentos, con aviso pasado el umbral. Ningún proceso tiene estado final de fallo.

Hitos

Una rama y un PR por hito.

Hito	Contenido	Hecho cuando
H1 ✓	booking ampliado, con versión, eventos y outbox; shared alineado (PR #21)	Crear, modificar y cancelar publican su evento con la versión correcta

Hito	Contenido	Hecho cuando
H2 ✓	<code>integration-model</code> , <code>partners</code> , <code>crs-integration-service</code> con <code>inbox</code> , relectura y router	Un cambio en <code>booking</code> arranca un proceso con la reserva canónica
H3 ✓	<code>mapping-service</code> con el diccionario, <code>Preparar</code> , causas y reanudación; UI y MCP	Una reserva con un código sin mapear se suspende y se reanuda al aprobarlo
H4 ✓	<code>opera-mock</code> , <code>pms-integration-service</code> contra él; definiciones #1 y #2	Punta a punta en local, con la guarda de secuencia
H5	Tenant real de Opera: aplazado el 2026-09-22 y cubierto por H12 (el conector escribe en XMAR desde el 2026-09-23)	—
H6 ✓	Proceso #3 contra el doble	Una reserva que referencia un interlocutor nuevo espera y se proyecta tras sincronizarlo
H7 ✓	<code>communication-service</code> y avisos	Cada tipo de aviso llega por email
H8 ✓	Propuesta de mapeado por agente	Desde la UI o el chat, el agente registra una propuesta que se aprueba y reanuda procesos
H9 ✓	Despliegue en el clúster, e2e y conclusiones (desplegado; conclusiones)	Demo en <code>ec1.mateu.io</code> ; conclusiones y coste cerrados
H13 ✓	El front office del hotel (<code>front-office</code> , traído de la demo de Mateu): cada reserva de MRU01 que se graba en Opera se graba también allí como estancia por llegar, y su cancelación la cancela; UI en <code>front.ec1.mateu.io</code> tras Keycloak	Una reserva de <code>ec1</code> está en XMAR y en el front office en 20 s
H14 ✓	La integración pms-fo (cadena CRS → PMS → front office): el front office se alimenta de lo que Opera tiene, no de lo que se le mandó; su propia integración en <code>integrations-service</code> (puertas, avisos, backfill, activación, sondeo de cambios); fuera los pasos <code>write-front-office</code> y <code>cancel-front-office</code> de las proyecciones	Una reserva de <code>ec1</code> llega a XMAR y de XMAR al front office con los nombres de Opera; un cambio en Opera llega por el sondeo
H12 ✓	El conector contra el tenant real (OHIP UAT, propiedad XMAR): diferencias con las specs corregidas en el conector y en el doble; interlocutores importados de Opera al ERP en vez de proyectados; en <code>ec1</code> , MRU01 integrado con XMAR	Una reserva de <code>ec1</code> llega a XMAR (13 s); falta en el tenant: cajero para depósitos e interfaces para referencias en perfiles
H11 ✓	<code>customer-mdm-service</code> con Salesforce (HLA CRM-MDM): identidad al proyectar, limpieza y fusión en Salesforce, supervivencia y propagación. En local contra la org real; desplegado en <code>ec1</code> (hoy 0.37.x, con marcado por calidad del dato y eventos por Pub/Sub con redes diarias, PR #133)	Una fusión hecha en Salesforce llega al perfil de Opera de las reservas del cliente absorbido
H10 ✓	<code>integrations-service</code> : la integración de cada hotel (conexión con Opera, secreto cifrado) y su alta por puertas como proceso <code>alta-integracion</code> ; el tráfico de un hotel sin integración activa espera	El alta de un hotel lleva sus reservas a Opera por backfill y la activación libera lo retenido

Después de H14 (sin número de hito)

Lo que se ha ido añadiendo tras los hitos, cada cosa en su PR y desplegada en `ec1` salvo que se diga:

Qué	PR
Motor EventConductor 2.23.1 sin parches; contratos por contexto con esquemas por topic y comprobación de tareas (<code>check-contracts.sh</code>)	#97, #100
Recepción hasta el PMS: check-in, check-out (con la factura de Opera) y no-show por el motor, con lo que XMAR pide de cada uno	#128, #132
Avisos de recepción de un cliente: Salesforce → MDM → front office; se leen antes del check-in y del check-out	#126, #131
Los cargos de recepción al folio de Opera y sus anulaciones; el total incluye los paquetes que Opera carga aparte	#135–#138, #141, #142
Habitación lista: Opera dice si la asignada está libre e inspeccionada; «Comprobar» en su tarjeta, aviso solo si no lo está	#135, #140
Check-in forzado: incompleto hasta completarlo, check-out bloqueado, aviso a recepción a las 24 h	#143, #144, #146, #148
MDM: contactos marcados por calidad del dato, limpieza solo de nombre, eventos de Salesforce por Pub/Sub con redes diarias	#133, #134
Agente de mapeado: propone todos los códigos pendientes; «Ask the agent» sin filtro pide para cada hotel con pendientes	#121, #129
IA: agentes que llaman a otros por A2A, guardarraíles de entrada y salida por ruta, chat del control plane con su propio agente	#111, #122
Consumo de las APIs de Salesforce y Opera: KPIs en las homes, dashboard <i>External APIs</i> y alertas en Grafana	#112, #116
Documentación del proyecto en <code>doc.ec1.mateu.io</code> (Starlight, en español, con usuario y contraseña)	#118
Avisos de reserva y de agencia: servicio <code>notices</code> en el plano de datos; el front office los aplica junto a los del cliente	#145 (desplegándose)
Auditoría de recepción y del CRS , con el «Historial» de la reserva	#147 (desplegándose)
Mateu 3.0-alpha.373 → 376 (Redwood: markdown del chat, acciones de página, confirmaciones, formularios en diálogo)	#110, #119, #139

Previsto, sin hacer: un solo despliegue de `ia-agent` para todos los canales. Hoy hay tres (`ia-agent` , `ia-agent-front-office` , `ia-agent-control-plane`) que solo difieren en el agente por defecto; el plan es que el gateway ponga en cada petición el canal y el agente por defecto según el host, y que las reglas de ruta del catálogo decidan con ellos.

Pendiente de recibir

1. **Tenant de Opera.** Recibido y en uso desde H12: gateway UAT (`mtce13ua` , `eu-frankfurt-1`), enterprise **RIUE**, cadena **RIUC**, propiedades **XMAR** (la de la PoC) y XMU. El usuario de integración no tiene cajero propio (FOF00094): se creó por OHIP un cajero para la PoC en XMAR, 69721441 «EC-DEMO1 Integración» (PR #125), que el conector pasa en check-outs y folios (`OPERA_CASHIER_ID`); los depósitos siguen apagados (`OPERA_POST_DEPOSITS=false`). Siguen sin configurar las interfaces para las referencias externas en perfiles (`OPERA_PROFILE_REFERENCES=false`). Las credenciales no van en el repo: van a `deploy/.secrets/` y de ahí a un Secret. **Rotar el client secret al cerrar la PoC.**

2. **Configuración del hotel de pruebas:** - Qué catálogos tiene y si se pueden leer por API. - Si hay **UDF libres** en la reserva para la guarda de secuencia. - Si hay perfiles de interlocutor ya creados.

Entregables

- El código de los hitos, en PRs.
- [cost-log.md](#), con el coste del conector OHIP separado del resto.
- **Conclusiones:** las incógnitas del HLA que la PoC cierra (R28, R38, la suspensión por causa sobre el motor), las que solo puede cerrar el tenant real (R18, R19, revalidación con `rsv`) y los problemas nuevos que haya sacado a la luz, como entrada para el DT.